



LeetCode 136: Single Number

1. Problem Title & Link

Title: LeetCode 136: Single Number

Link: <https://leetcode.com/problems/single-number/>

2. Problem Statement (Short Summary)

Given a non-empty array of integers.

Every element appears **twice** except for one element which appears exactly **once**.

Find and return that single element.

You must solve it with:

$O(n)$ Time

$O(1)$ Extra Space

Example

Input:

nums = [2,2,1]

Output:

1

Explanation:

2 appears twice

1 appears once

3. Examples (Input → Output)

Example 1

Input:

nums = [2,2,1]

Output:

1

Example 2

Input:

nums = [4,1,2,1,2]

Output:

4

Example 3



Input:

nums = [1]

Output:

1

Example 4

Input:

nums = [7,3,5,4,5,3,4]

Output:

7

4. Constraints

$1 \leq \text{nums.length} \leq 3 \times 10^4$

$-3 \times 10^4 \leq \text{nums}[i] \leq 3 \times 10^4$

Important:

Every element appears twice
except one.

5. Core Concept (Pattern / Topic)

Bit Manipulation (XOR)

Secondary Topics:

- HashMap
- Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Count frequency of every element.

Example:

[4,1,2,1,2]

Frequency:

4 → 1

1 → 2

2 → 2

Answer:



4

Works.

Complexity

Time : $O(n)$

Space : $O(n)$

Uses HashMap.

Better Observation

We need:

$O(1)$ space

HashMap is not allowed.

XOR Properties

Property 1

$$a \oplus a = 0$$

Example:

$$5 \oplus 5 = 0$$

Property 2

$$a \oplus 0 = a$$

Example:

$$7 \oplus 0 = 7$$

Property 3

XOR is commutative

$$a \oplus b \oplus a$$

=

$$a \oplus a \oplus b$$

=

$$0 \oplus b$$

=



b

Key Insight

All duplicate numbers cancel each other.

Only the single number remains.

Example

[4,1,2,1,2]

XOR all values:

$4 \wedge 1 \wedge 2 \wedge 1 \wedge 2$

Rearrange:

$4 \wedge (1 \wedge 1) \wedge (2 \wedge 2)$

$4 \wedge 0 \wedge 0$

4

Answer:

4

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
[4,1,2,1,2]
4 ^ 1 ^ 2 ^ 1 ^ 2
Group duplicates:
4 ^ (1^1) ^ (2^2)
4 ^ 0 ^ 0
4

Cancellation View
1 ---- 1

2 ---- 2

Disappear

Only:

4
remains.
```

8. Pseudocode (Language Independent)

```
result = 0

for each number
```



```
        result = result XOR number

return result
```

9. Code Implementation

Python

```
class Solution:
    def singleNumber(self, nums):

        result = 0

        for num in nums:
            result ^= num

        return result
```

Java

```
class Solution {

    public int singleNumber(int[] nums) {

        int result = 0;

        for(int num : nums) {

            result ^= num;
        }

        return result;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

Space Complexity

$O(1)$

Only one variable used.

11. Common Mistakes / Edge Cases

**Mistake 1**

Using addition.

Wrong:

$2+2+1$

does not eliminate duplicates.

Mistake 2

Using HashMap.

Works but violates:

$O(1)$ Space

requirement.

Edge Cases**Single Element**

Input:

[5]

Output:

5

Negative Numbers

Input:

[-1,-1,-2]

Output:

-2

XOR still works.

12. Detailed Dry Run

Input:

nums = [4,1,2,1,2]

Initial:

result = 0

Number	XOR Operation	Result
4	$0 \oplus 4$	4
1	$4 \oplus 1$	5
2	$5 \oplus 2$	7
1	$7 \oplus 1$	6



2	$6 \wedge 2$	4
---	--------------	---

Final:

4

Return:

4

13. Common Use Cases (Real-Life / Interview)

Error Detection

XOR is heavily used in:

Networking

Checksums

Data Validation

Cryptography

Simple encryption concepts.

Memory Optimization

Finding unique values without extra storage.

Interview Purpose

Tests:

Bit Manipulation

XOR Properties

Mathematical Thinking

14. Common Traps (Important!)

Trap 1

Not knowing XOR rules.

Remember:

$$a \wedge a = 0$$

Trap 2

Thinking Bit Manipulation is difficult.

This is one of the easiest XOR applications.

Trap 3

Trying sorting.



Sorting gives:

$O(n \log n)$

while XOR gives:

$O(n)$

15. Builds To (Related LeetCode Problems)

LC 136 - Single Number



LC 268 - Missing Number



LC 191 - Number of 1 Bits



LC 338 - Counting Bits



LC 137 - Single Number II



LC 260 - Single Number III

These build strong Bit Manipulation skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Nested Loops	$O(n^2)$	$O(1)$	Poor
Sorting	$O(n \log n)$	$O(1)$	Fair
HashMap	$O(n)$	$O(n)$	Good
XOR	$O(n)$	$O(1)$	Best

Approach 1: HashMap

Count frequencies.

Simple but uses extra memory.

Approach 2: XOR (Chosen)

Most elegant.

Interview favorite.

17. Why This Solution Works (Short Intuition)



Every duplicate number appears exactly twice.

Using XOR:

$$a \oplus a = 0$$

all duplicate pairs cancel out, leaving only the element that appears once.

18. Variations / Follow-Up Questions

Follow-Up 1

Every element appears 3 times except one.

Related:

LC 137

Follow-Up 2

Exactly two unique elements exist.

Related:

LC 260

Follow-Up 3

Find missing number using XOR.

Related:

LC 268

Follow-Up 4

Find element occurring odd number of times.

XOR works directly.

Interview Takeaway

This is one of the most important XOR interview problems.

The entire solution relies on:

$$a \oplus a = 0$$

$$a \oplus 0 = a$$

Whenever you hear:

Every element appears twice

except one

immediately think:

XOR

This pattern frequently appears in Zoho, Amazon, Microsoft, and product-company coding rounds.

